Assignment 5: Lagrangian Dynamics

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski
Albert Xu
Henry Liao
Karthik Srinivasan

Contents

1 Code Questions 3

2 Hands-on Lab: Drawing with the xArm7 4
2.1 The xArm7 as a Planar Arm 4
2.2 Setting Up 4

3 Submission Checklist 8

1 Code Questions

1) Double Pendulum Simulation *40 points*

In this question you will simulate the double pendulum using the equations of motion you derived previously. All the code is in the `code/` folder; `code/README.md` explains how to run each script. You edit and submit only `double_pendulum.py`.

Use the same conventions as your derivation: $q = [\theta_1, \theta_2]^\top$ with both angles measured from the downward vertical (counterclockwise positive), θ_2 the absolute angle of link 2 rather than its angle relative to link 1, and point masses m_1, m_2 on massless links ℓ_1, ℓ_2. There is no applied torque in the code ($\tau = 0$). In your code, use the constants `M1`, `M2`, `L1`, `L2` and `G` at the top of `double_pendulum.py` rather than writing the numbers in directly, because the autograder changes them.

a. [20 points] Implement `dynamics(q, qd)`, which returns the joint accelerations $\ddot{q}$ as an array of shape $(2,)$. Your Euler-Lagrange equations are linear in $\ddot{q}$, so write them as $M(q)\ddot{q} = b(q, \dot{q})$ and solve for $\ddot{q}$ with `np.linalg.solve`.

b. [15 points] Implement `simulate(q0, qd0, T, dt)`. We recommend using a library like `scipy.integrate.solve_ivp` to integrate the equations of motion with the supplied `RTOL` and `ATOL` tolerances, but also encourage you to try implementing your own integrator to learn a little about numerical integration. You'll see the difference in (c)'s energy check.

The output format should be a tuple `(t, q, qd)` of numpy arrays, sampled every `dt` seconds from 0 to T inclusive (use `np.arange(0, T + dt / 2, dt)`, so there are $N = T/\Delta t + 1$ samples):

- `t`: the sample times $0, \Delta t, 2\Delta t, \ldots, T$, shape $(N,)$.
- `q`: the joint angles $[\theta_1, \theta_2]$ at each sample time, shape $(N, 2)$.
- `qd`: the joint velocities $[\dot{\theta}_1, \dot{\theta}_2]$ at each sample time, shape $(N, 2)$.

The first row of `q` and `qd` is the initial state `q0`, `qd0`.

Run `python local_autograder.py` to check parts (a) and (b), then watch your pendulum with `python visualize.py`.

c. [5 points] With no friction and no applied torque, the total energy $E = T + V$ of the double pendulum stays constant. That gives you a way to check a simulation without knowing the right answer: if E changes over time, the simulation is not exactly following the physics.

Implement `total_energy(q, qd)` in `double_pendulum.py` using your T and V from your derivation; the autograder checks it. Then run `python energy_check.py`, which simulates the pendulum for 20 seconds and plots its total energy over time. With `solve_ivp` at the supplied tolerances the line should be flat; if you wrote your own integrator, see how far it drifts.

Try it (not graded). Run `python visualize.py --chaos 5 --plot`, which simulates five pendulums whose starting θ_2 values differ by only 0.001 rad, and watch how long they stay together. Then try a small initial angle, `python visualize.py --chaos 5 --q0 10 5` and compare.

2 Hands-on Lab: Drawing with the xArm7

In this section the xArm7 holds a marker and writes on a sheet of paper. You already have most of the pieces: your Assignment 4 `Robot` does the inverse kinematics, and the arm is driven exactly as you drove planar chains there. What is new is where the paper is. You will find it the way a technician would: touch the marker to its four corners, and turn those four points into a coordinate frame for the sheet. Then you will draw the letters **RKD** on screen, run them in simulation, and finally watch the real arm write them.

2.1 The xArm7 as a Planar Arm

Hold joints 3 and 5 at zero and every joint of the xArm7 lies in one vertical plane, the one joint 1 turns. Joints 2, 4 and 6 are then three parallel hinges in that plane: a planar RRR arm standing up, with the marker on the end of its third link. To put the marker tip on a point $\mathbf{p} = [p_x, p_y, p_z]^\top$ in the robot's base frame,

1. joint 1 turns to face it, $q_1 = \operatorname{atan2}(p_y, p_x)$;
2. in that plane, $\mathbf{p}$ is at $r = \sqrt{p_x^2 + p_y^2}$ out and $z = p_z - 0.267$ m up from the shoulder;
3. your `Robot.inverse_kinematics` solves the RRR for the goal $[r, z, \phi]$, where ϕ is the angle of the third link that keeps the marker pointing into the paper.

`lab/drawing_arm.py` does steps 1 and 2 and calls your IK for step 3. It measures the three link lengths and the joint offsets from UFACTORY's own model of the arm, so you don't have to. Your IK takes 100 steps of size 0.1, which was sized for the metre-long links of Assignment 4. The xArm's links are about a third of that, so one call only gets part of the way, and `drawing_arm.py` calls it again from its own answer until the marker lands within 1 mm.

The marker always points straight down, and joint 7 turns against joint 1 so that the tool never rotates about the vertical either. The tool only ever translates: in calibration and in drawing alike, it keeps one fixed orientation. That is what makes the drawing independent of the marker. The code computes the tip as $\mathbf{t} = \mathbf{f} + \mathbf{R}\,[0, 0, \ell]^\top$ from the flange position $\mathbf{f}$ and orientation $\mathbf{R}$, with ℓ the nominal `MARKER_LENGTH`; the real tip is $\mathbf{f} + \mathbf{R}\,\mathbf{d}$ for some $\mathbf{d}$ nobody measured. With $\mathbf{R}$ fixed, the two differ by the same vector everywhere, so the calibration, made with the computed tip, finds the paper shifted by that vector, and the drawing, planned with the computed tip, puts the real tip back on it. (The simulator's marker is 18 mm longer and 5 mm off-axis compared with `MARKER_LENGTH`, and the ink still lands within about a millimetre.) It is also why you never push the marker onto the paper by hand: a hand would hold it at a different angle each time.

2.2 Setting Up

The lab uses the course robot library, `xarm7_lib`, the same one as the Assignment 3 lab. Update it before you start:

```
git clone https://github.com/CMU-16384-2026-Fall/16384-robot-lib
cd 16384-robot-lib
```

```
git pull                          # if you already had it cloned
conda env update -f environment.yml
conda activate 16384
```

Run every command in this section from the `lab` folder. As before, the same scripts drive the real arm when the `ROBOT_IP` environment variable is set, and a MuJoCo simulation, shown in your browser, when it is not. Everything up to the last part can be done at home.

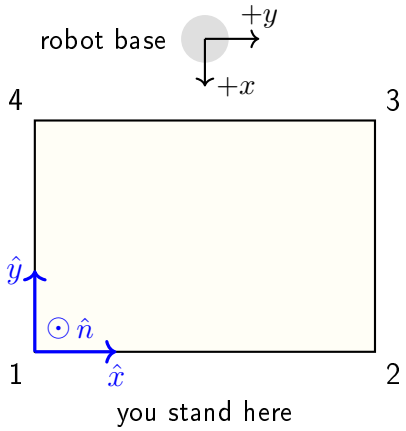


Figure 1: The sheet from above. You touch corners 1–4 in order; the paper frame has its origin at corner 1, $\hat{x}$ towards corner 2, and $\hat{n}$ out of the paper.

2) Drawing RKD on the xArm7 *50 points*

This question is graded on your `plane.py` (autograded) and on one video of the real arm. There is nothing to write up.

a. [10 points] **The paper's frame.** Calibration gives you four touched corners $\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4$ in the robot's base frame, in the order of Figure 1. They will not lie exactly on one plane: the paper isn't perfectly flat, and each touch is only as good as your eye. So the frame uses all four:

- the unit normal from the cross product of the diagonals, $\hat{n} = \mathbf{n}/\|\mathbf{n}\|$ with $\mathbf{n} = (\mathbf{p}_3 - \mathbf{p}_1) \times (\mathbf{p}_4 - \mathbf{p}_2)$; it is perpendicular to both diagonals, weighs all four touches alike, and with the corners in this order points up out of the paper;
- the plane through the centroid $\mathbf{c} = \frac{1}{4}\sum_i \mathbf{p}_i$;
- the origin $\mathbf{o}$: corner 1, moved along $\hat{n}$ onto that plane;
- the x axis $\hat{x}$: $\mathbf{p}_2 - \mathbf{p}_1$ with its component along $\hat{n}$ removed, normalized; and $\hat{y} = \hat{n} \times \hat{x}$.

Work out the formulas for $\mathbf{o}$ and $\hat{x}$, and implement them in `lab/plane.py`: `plane_frame` returns the 4×4 homogeneous transform

$$\mathbf{H} = \begin{bmatrix} \hat{x} & \hat{y} & \hat{n} & \mathbf{o} \\ 0 & 0 & 0 & 1 \end{bmatrix};$$

`paper_to_robot` maps a point (u, v) on the paper, at height h above it, to $\mathbf{H}\,[u, v, h, 1]^\top$; and `robot_to_paper` is its inverse, written without a matrix inverse. These three are

autograded, on randomly perturbed corners as in Question 1. Then check them before going anywhere near the arm:

`python check_plane.py`

It tries your frame on sheets whose answer is known (flat, turned, tilted, a whiteboard standing up), on noisy touches checked against every property the formula promises, and on the graded cases, the same ones `local_autograder.py -p plane` runs from `code/`. Every failure says what is wrong. A wrong frame sends the marker into the table, so do not take the lab past this point until it reports no failures.

b. **Draw it, in simulation** (not graded, but do it before your lab session).

(i) Copy your Assignment 4 `robot.py` over `lab/robot.py`, whole and unchanged, for example `cp ../../Assignment4/ex2/robot.py robot.py` (the `robot.py` in `ex1`, `ex2` and `ex3` is the same class; take the one with your IK in it). The forward kinematics, Jacobian and gradient-descent IK you already wrote are what the simulated arm runs. (The real sheet can sit lower than the planar chain reaches, so on the real arm `draw.py` uses a 7-joint IK of ours that keeps joints 3 and 5 at zero wherever it can: where your IK reaches, it finds the same answer.) If it is still the stub, or the IK isn't filled in, the lab scripts say so before anything moves.

(ii) Calibrate the simulator's sheet: `python calibrate.py`. The simulator has a sheet of paper of its own, placed a little crooked on purpose. It touches the four corners with your IK, builds the frame with your `plane.py`, and tells you how far your frame is from the sheet's true one. It should be within a few millimetres. Then practise touching the corners: `python calibrate.py --manual` has you jog the marker onto each corner with the keyboard (the keys are in part (c)), while it tells you how far the tip is from the paper.

(iii) Draw the strokes for the letters **RKD**: `python sketch.py strokes/rkd.json`. Left-click to place points, right-click to finish a stroke; the window lists the other keys. Each stroke is traced without lifting the marker, so think about where to put the breaks. Closing the window saves the file. The strokes are stored in metres on the sheet, from corner 1, so the same file draws in simulation and on the real arm.

(iv) Run it: `python draw.py strokes/rkd.json`. Your IK is solved along the whole path first, which takes a few seconds, and any point it can't reach stops the run before anything moves. Then the simulated arm writes on its sheet, and ink appears where the marker meets the paper. The result is saved as `results/rkd-sim.png`: your strokes, dashed, beside the path the marker really took.

c. [40 points] **Draw it on the xArm7.** Graded by hand, from your video. The sheet is taped to the table for you, about 50 cm straight out from the base; don't move it. In lab, with `ROBOT_IP` set:

(i) `git pull` your code onto the lab machine, and run `python check_plane.py` once more.

(ii) `python calibrate.py`.

- Before the arm moves, its gripper opens (hold any marker already in it). Put the marker between the fingers, tip down and straight, press enter, hold it until

they close on it, and let go. It must not move in the gripper from here on: a marker put back in means calibrating again. (To calibrate again with the same marker still in, run `python calibrate.py --keep-marker`.)

- The arm moves to the home pose of the Assignment 3 lab, and every joint is let go. For each corner in turn, bring the marker over the corner, down to a centimetre or two above it, and press enter. It only has to be close. If you take the marker out of the area the arm can draw on, the arm locks where it is, asks you to let go, and moves back.
- For each corner, the arm puts the marker upright a little over it and lets go again. Put the tip on the corner, keeping the marker upright, and press enter. The script says how far the marker leant; keep it under 5 degrees, since the marker's offset only cancels if it is upright at every touch, as it is while drawing. The arm then takes the marker to the next corner, never lower than the paper.

Any order round the sheet works: the corners are put in order for you. The controller's own safety fence is off while calibrating and back on when it ends; the library's safety box stays on throughout. The simulator has no free drive, so there `calibrate.py --manual` has you make the touches with the keyboard instead (`w`/`s`, `a`/`d`, `r`/`f`, with `1`, `2`, `3` setting the step to 0.5, 2 or 10 mm).

(iii) `python draw.py strokes/rkd.json --plan-only` plans the drawing on your real sheet without moving anything. Look at `results/rkd-plan.png`.

(iv) `python draw.py strokes/rkd.json`, hand on the e-stop.

Safety: Read this before you touch the robot

- **The e-stop.** Know where it is before you start, and keep a hand on it while the arm draws. Press it for anything you did not expect. Stopping the arm costs you nothing.
- **Stay out of the workspace** while the arm moves under its own power. The only time you touch it is during calibration, when the script says the arm is free.
- **"Press enter when your hands are clear" means it.** Every move the scripts make on their own waits for that prompt.
- **The safety box** is the one from Assignment 3: $x \in [-0.10, 0.70]$, $y \in [-0.45, 0.45]$, $z \in [0, 0.80]$ m. `draw.py` checks the whole drawing against it, and against self-collision, before anything moves. It does *not* know about the marker, so the marker is yours to keep off the table: that is what the calibration is for.
- **If the marker digs in**, press the e-stop. The drawing presses 1 mm below the surface you touched, which a sprung marker holder absorbs. More than that means the calibration is off: calibrate again.

Deliverable. One video, a phone is fine, showing the arm write RKD from start to finish, with the paper visible. Put it in `lab/handin/`. `create_submission.py` packages it with your `lab/robot.py`, `lab/plane.py`, your strokes, your calibrations and `lab/results`.

3 Submission Checklist

- ☐ Run `local_autograder.py` and `lab/check_plane.py`, and confirm they report no failures.
- ☐ Put your drawing video in `lab/handin`.
- ☐ Run `python create_submission.py` from the `code` folder.
- ☐ Rename `submission.zip` and upload `<andrew_id>.zip` to Gradescope. There is no PDF to submit.
- ☐ The code is autograded, with no limit on number of submissions within the mandated deadline.